
COMP206 Learning Journal

Hector Barquero

Jul 26, 2026

CONTENTS:

1	Assignment four journal	1
1.1	Program design	1
1.2	Program test plan	4
1.3	Overview and reflections	5
1.4	Research notes	7
1.5	Sources and references	8

ASSIGNMENT FOUR JOURNAL

Time spent	Jan 2026 to Aug 2026
Student name	Hector Barquero

 Tip**How to use this journal**

Learning journal contains program design notes, changelogs for versions as I made updates, reflections, and reading notes for the units of each assignment. There's also practice questions from the textbook which were recommended to be added to the learning journal for each assignment chapter.

1.1 Program design

1.1.1 Concept and scope

This project is a compact parser-based adventure game inspired by Lewis Carroll's *Alice's Adventures in Wonderland*. You (the player) represent Alice and begin beside the rabbit hole. The White Rabbit has lost four important objects: the Golden Key, Pocket Watch, Painted Rose and Queen's Tart. You must descend into Wonderland, explore its connected locations, collect the four treasures and return them to the Hall of Doors. The Hall is the game's safe room. After all treasures are stashed there, Alice must use the DRINK ME Bottle to become small enough to enter the tiny door and reach the Quiet Garden.

The game uses recognizable characters, places and objects while giving them a new game objective. The game scope demonstrates object-oriented design, external data loading, a command parser, state changes, inventory management, random events, scoring, winning and losing. It remains small enough to test every location and important command manually and I've included some instructions for fast tracking to certain winning or losing states.

1.1.2 Player experience and command style

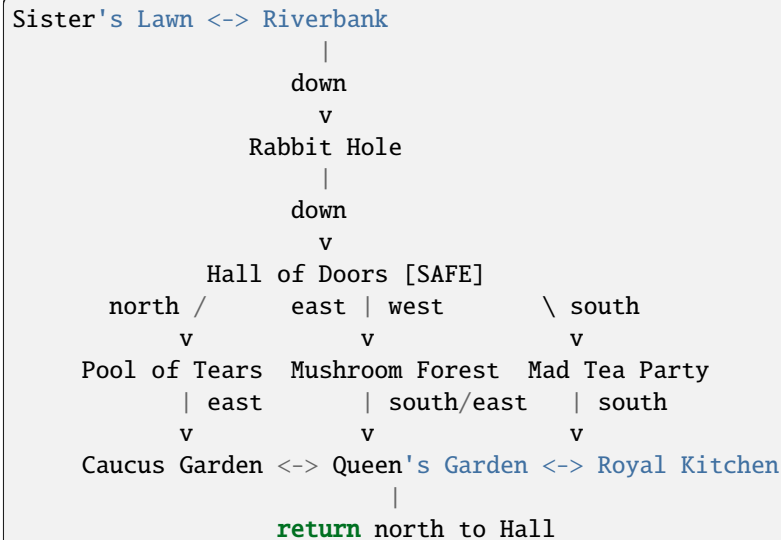
All communication is text. On the first visit to a location, the game prints a long description. Later visits print a shorter reminder. It then lists visible items, characters and available exits. The player enters short commands. Input is converted to lowercase, punctuation becomes whitespace and repeated spaces are removed. Therefore GO NORTH!, go north and Go, north are treated consistently.

Direction shortcuts are supported. go north, north and n all produce the same internal command. Other aliases are loaded from actions.txt. Examples include inventory, invent and i or take, get and pick up. The parser separates an action from its object and optional target. For example, throw cup at hatter becomes the action throw, object cup and target hatter.

The opening message states the goal. `help` lists valid commands and `score` shows score, turns, energy, courage and treasure progress. Invalid commands do not end the game. They produce a direct message and allow another attempt.

1.1.3 World plan

The map contains eleven locations. The rabbit hole is intentionally one-way. Once Alice moves down from the Riverbank, no exit leads back above ground.



Hall of Doors --down, when unlocked and Alice is small--> Quiet Garden [WIN]

The Hall of Doors has five exits, satisfying the requirement for a room with more than three exits. It connects the major areas and acts as a safe gathering point. Stashed items do not count against the six-item carrying capacity. This encourages the player to return rather than carrying every useful object at once.

The locations are Sister's Lawn, Riverbank, Rabbit Hole, Hall of Doors, Pool of Tears, Caucus Garden, Mushroom Forest, Mad Tea Party, Queen's Garden, Royal Kitchen and Quiet Garden. Every location has an outgoing route and at least one incoming connection. The Rabbit Hole prevents reverse travel by omitting an upward exit. The Quiet Garden has an exit in the data model, although reaching it immediately completes the game.

1.1.4 Items and treasures

The game defines 16 interactive items, exceeding the required 12. They include the White Rabbit's Fan, Marmalade Jar, DRINK ME Bottle, EAT ME Cake, Thimble, Box of Comfits, Golden Key, Mushroom Piece, Tea Cup, Pocket Watch, Bread and Butter, Painted Rose, Croquet Flamingo, Hedgehog Croquet Ball, Pepper Shaker and Queen's Tart.

Items have an identifier, display name, aliases, description, current location, takeable flag, treasure flag, point value, use message and state flags. Their location changes to `inventory` when taken, a room identifier when dropped, `stash` when stored safely or `consumed` after single-use food and drink. This produces one consistent representation instead of separate room and inventory copies.

Food restores energy. The fan and painted rose protect Alice during hostile encounters. The pocket watch reports the current turn. The flamingo and hedgehog can be combined indirectly by carrying both and using either on the croquet field. The DRINK ME Bottle sets the small-size state needed for the final door. The EAT ME Cake and Mushroom Piece restore ordinary size.

1.1.5 Characters and encounters

Eight characters are defined: White Rabbit, Dodo, Caterpillar, Mad Hatter, Dormouse, Cheshire Cat, Queen of Hearts and Duchess. Each stores aliases, a location, description, friendly flag, dialogue and random encounter text. Talking to a character prints location-specific dialogue and awards a small first-conversation score.

Random encounters can occur after state-changing actions. A friendly encounter adds atmosphere without punishment. A hostile Queen or Duchess encounter may reduce courage. Carrying the tart makes a Queen encounter more likely. The fan or painted rose grants temporary protection. The `throw` action also has a random outcome: the target either ducks or is bumped harmlessly. Random events use a Mersenne Twister generator. A seed can be supplied for repeatable testing, and random encounters can be disabled when testing a fixed route.

1.1.6 Classes and responsibilities

`Game` is the highest class. It owns the repositories, current location, `Inventory`, `Control` object, score, turn count, energy, courage and other game flags. It loads data, runs the loop and dispatches parsed commands to methods.

`Location` stores one room. `Locations` loads and manages all `Location` objects. Each `Location` contains descriptions, visited and safe flags, and a map from directions to destination identifiers.

`Item` stores one object. `Items` loads all `Item` objects and finds them by identifier, alias and current location. `Character` and `Characters` use the same entity-and-repository pattern for non-player characters.

`Action` stores a canonical action, aliases and help text. `Actions` loads the action vocabulary and finds the longest alias matching the beginning of a command. Longest matching is necessary so `pick up` is selected as a phrase instead of treating only `pick` as an action.

`Inventory` maintains a vector of carried item identifiers and enforces a capacity of six. It does not duplicate full `Item` objects. `Control` reads a line, normalizes it, recognizes direction shortcuts, matches a loaded `Action` and separates the object and target. `TextUtils` supplies reusable string operations such as trimming, splitting, joining and command normalization.

1.1.7 External text files

`locations.txt` contains room names, long and short descriptions, the safe flag and exits. `items.txt` defines object properties and starting locations. `characters.txt` contains descriptions, dialogue and encounter text. `actions.txt` contains canonical command names, aliases and help descriptions. Each non-comment record uses vertical bars between fields and commas inside alias or exit lists. Loading fails with a specific message if a file is missing or a record has the wrong number of fields. Cross-references are checked so an item, character or exit cannot silently point to an unknown location.

1.1.8 Game state, scoring and ending

A successful action advances the turn. Energy decreases every third turn. Consumable items restore it. If energy reaches zero, the game ends. Alice begins with three courage points. Hostile interactions and some random encounters can reduce courage; reaching zero also ends the game.

The player earns points for first visits, first conversations, selected items, the croquet interaction and safely stashing treasures. The score is secondary to the explicit win condition. The tiny door checks two facts: all four required treasures currently have `stash` as their location, and Alice has used the DRINK ME Bottle. Passing both checks allows travel to the Quiet Garden, where the game prints the final score and ends successfully.

1.1.9 Algorithmic scale

Command normalization is linear in command length, $O(c)$. Action matching is $O(a * s)$ for a aliases and short command strings. Item and character searches are linear in their small collections, $O(i)$ and $O(k)$. Location lookup and exit lookup use ordered maps, giving $O(\log r)$ for r rooms or exits. The complete game loop is $O(t * p)$, where

t is the number of turns and p is the bounded per-command parsing and object-search work. With the fixed assignment data, each turn behaves as constant practical work.

1.2 Program test plan

```
g++ -o Wonderland.exe *.cpp
```

Expected result: compilation completes without warnings or errors. Run with ./Wonderland.exe

```
./Wonderland.exe --seed 1 --no-random < tests/win_commands.txt
```

Expected results:

- The program loads all four external data files.
- The four treasures can be collected and stashed.
- The output states that all four treasures are safe.
- Using the bottle enables the final downward movement.
- The Quiet Garden and You win messages appear.

```
./Wonderland.exe --seed 1 --no-random < tests/parser_commands.txt
```

Expected results:

- Punctuation and capitalization are ignored.
- `invent` displays the inventory.
- An unavailable direction is rejected.
- An unknown action is rejected without terminating the loop.
- Taking and dropping the fan changes inventory state.

```
./Wonderland.exe --data tests/missing-data
```

Expected result: the program reports the missing `locations.txt` file and returns a failure status.

1. Fill all six inventory slots and attempt to take another item. Confirm that the capacity message appears.
2. Attempt `stash key` outside the Hall of Doors. Confirm that its rejected.
3. Stash an item in the Hall, then take it again. Confirm that it returns to inventory.
4. Attempt to enter the tiny door without four treasures. Confirm its locked.
5. Stash all treasures but do not use the bottle. Confirm Alice is too large.
6. Use the cake after the bottle. Confirm Alice is again too large for the door.
7. Use `talk to queen` without protection until courage reaches zero. Confirm the loss condition.
8. Use food while energy is below its maximum. Confirm recovery does not exceed 30.
9. Carry the tart with random encounters enabled and a fixed seed. Confirm a Queen encounter can occur.
10. Change an item location to an unknown ID in `items.txt`. Confirm startup validation reports the item ID.

1.3 Overview and reflections

I began by reviewing the history and interaction style of *Colossal Cave Adventure*. The research showed that the game established the familiar pattern of a narrator describing a location, a player entering short commands, and the program changing a model of rooms, objects and progress. Sources describing the game emphasize movement between named locations, contextual object commands, inventory, treasure collection and a score. I also noted the early parser's separation between motion words and action words. In particular, historical versions treated commands such as `go north` and `north` as equivalent. That directly influenced the command handling in this project.

The Ubuntu manual page for Open Adventure was useful because it describes the parser behaviour from a player's perspective. It identifies the historic motion and action distinction and explains that motion words are handled immediately. I did not reproduce that parser exactly because some of its odd interpretations would be confusing in a new student project. Instead, I kept the useful conventions: short commands, action-object structure, direction abbreviations and forgiving capitalization.

I read the Project Gutenberg edition of *Alice's Adventures in Wonderland* to select locations, characters and objects. The final design uses the riverbank, rabbit hole, hall of doors, pool of tears, caucus race, mushroom scene, tea party, Queen's garden and kitchen. Objects such as the fan, marmalade jar, labelled bottle and cake, golden key, mushroom, watch, painted rose, flamingo and tarts gave the game enough items to support inventory actions without inventing an unrelated fantasy setting.

The research also suggested separating program logic from descriptive data. Historical Adventure implementations contained substantial location, vocabulary and message data in addition to executable logic. I used that idea in a smaller object-oriented form: four text files define the world while C++ classes implement the rules. This also satisfies the assignment requirement that actions, items, characters and location descriptions be loaded at startup.

The first decision was to avoid adapting the entire book. That would create too many rooms and special cases to test reliably. I selected a connected group of recognizable scenes and gave them a new, explicit objective. The White Rabbit has lost four treasures. Alice must recover and store them, then enter the tiny door. The objective is stated before the first command so the player doesn't need prior knowledge of the novel.

I chose the Hall of Doors as the safe room because it's central to the selected scenes and naturally supports many doors. It has five exits, so it also satisfies the requirement that one room have more than three exits. The map branches from this hub into the Pool of Tears, Mushroom Forest, Mad Tea Party and Queen's Garden. Cross-connections reduce unnecessary backtracking and make the world feel less like a list of isolated rooms.

Inventory capacity is six. This is not required for basic storage, but it gives the safe room a practical purpose. Alice can carry all four treasures and one or two tools, but collecting every object at once is impossible. The player must choose, drop or stash. Items are represented by one object whose location field changes. I selected this model after considering separate lists in each room, the inventory and the safe area. Separate lists create synchronization problems because an item can accidentally exist in two places. A single location value makes the invariant clear: each item is in exactly one place.

The parser is deliberately limited rather than pretending to understand free English. It normalizes text and searches the loaded action aliases. The longest matching alias is selected, which permits a phrase such as `pick up`. The remaining words become the object, except that `at` and `on` can divide an object from a target. This supports commands such as `throw cup at hatter`. Direction words are handled separately so `go north`, `north` and `n` map to the same command.

I used first-visit and repeat descriptions to match a common text-adventure pattern and the assignment requirements. The Location object stores both forms and a visited flag. The first visit gives two exploration points and prints the long description. Later visits print the shorter version. The rule is contained in one method rather than repeated in every movement branch.

Random encounters were added only after deterministic movement and item logic worked. Random behaviour can make testing difficult, so the program supports a fixed seed and a `--no-random` option. Normal play still includes encounters. The Queen becomes more likely when Alice carries the tart, demonstrating that inventory can influence an

encounter. Hostile encounters reduce courage only sometimes, and protective items can prevent the loss. This keeps randomness meaningful without making a winning route depend entirely on luck.

The largest structural challenge was deciding how the required plural classes should interact. I created small entity structures, such as Location and Item, plus collection classes named Locations and Items. The collections are responsible for loading and searching, while Game is responsible for game rules. This prevents Game from parsing every file itself and keeps the exact required component names visible in the design.

External data parsing introduced several possible errors: missing files, wrong field counts, invalid numeric values and references to locations that do not exist. Initially it would have been possible for an item to load with a bad room identifier and simply never appear. I added validation after loading. Game checks the starting room, every item location, every character location and every exit destination. A failed check prints the responsible identifier and ends startup.

A specific identifier bug appeared during the first winning-route test. The same function used to normalize player commands was also used on data IDs. Because command normalization replaces punctuation with spaces, an identifier such as `hall_of_doors` became `hall of doors`. Hard-coded state checks still used the underscore form, so the final-door restrictions were bypassed and the win location was not recognized. I corrected the design by treating internal identifiers differently from commands. IDs are trimmed and converted to lowercase but retain underscores. Player aliases are still normalized aggressively. This test demonstrated why machine identifiers and user-facing text should have separate rules.

Another parser issue involved `use object with target`. The separators ``` on ``` and ``` with ``` have different lengths. The first implementation used the same substring offset for both, leaving part of the word `with` in the target. I changed the parser to record which separator matched and remove the correct number of characters.

C++ object ownership was kept simple. Game owns repository objects directly and owns Control through `std::unique_ptr` because Control cannot be constructed until Actions has loaded. Items and characters are stored as values in maps. Inventory stores only IDs, so it cannot own or duplicate an Item. Methods obtain pointers to repository values for short operations. No pointer is stored after the map changes, and the maps are not structurally modified after loading.

Compilation testing used C++17 with `-Wall`, `-Wextra` and `-pedantic`. The complete project compiled with no warnings. The first automated scenario uses a command transcript and disables random encounters. It travels from the riverbank through the rabbit hole, collects the four treasures, returns to the Hall of Doors, stashes them, uses the bottle and enters the Quiet Garden. The expected output includes the four-treasure message, the win message and a final score.

Parser tests include capitalization, punctuation and aliases. `LOOK!!!` must behave as `look`. `INVENT` must behave as `inventory`. Movement tests use both full and one-letter forms. Unknown actions must report an error without changing location. Invalid movement must also preserve the current location.

Inventory testing verifies take, list and drop. A separate manual sequence fills all six slots and attempts to take a seventh item. The expected response explains that something must be dropped or stashed. Safe-room testing verifies that `stash` fails elsewhere, succeeds in the Hall, removes the item from carried inventory and makes it visible in the safe-room list. Taking a stashed item back out reduces the current treasure count.

Item tests cover size and energy state. The DRINK ME Bottle is consumed once and sets the small state. The EAT ME Cake and Mushroom Piece restore normal size. Food increases energy without passing the maximum. The pocket watch reports the turn count. Carrying both croquet objects and using one in the Queen's Garden awards the croquet score only once.

Character tests call `talk` for friendly and hostile characters. A first conversation awards points, while repeated conversation doesn't repeatedly add that score. Hostile conversation reduces courage when protection is inactive. The fan or rose supplies temporary protection. Throw tests use a fixed seed to confirm both the parser and the item's transfer from inventory back to the room.

Failure testing renames or omits a data file and checks for a non-zero exit. I also changed an exit destination to an invalid identifier and confirmed that startup rejected it. These tests matter because the external text files are part of the final program, not optional supporting material.

The random system was tested in two modes. With a fixed seed, the same sequence of actions produces repeatable encounters. With a random-device seed, repeated short games produce different encounter timing. Randomness is intentionally not used for core puzzle success. A player can always win through correct movement, inventory and size state if energy and courage are managed.

The parser doesn't understand unrestricted sentences or pronouns. It expects a known action followed by a recognizable object name. This is appropriate for the course scope and is made explicit by the help command. The data format also doesn't support vertical bars inside descriptive text because that character is the field separator.

The final design meets the required minimums and provides a complete game loop. It includes eleven locations, eight characters, sixteen items, external data, random events, a safe room, inventory changes, long and short descriptions, multiple command forms, loss conditions and a defined win. The major design lesson was that a text adventure is easiest to manage when world data, parsing and state-changing rules have separate responsibilities.

1.4 Research notes

1.4.1 Research and game concept

- Researched the short-command parser, room exploration, inventory and treasure-scoring conventions used by Colossal Cave Adventure. (v0.1-v0.2)
- Reviewed Alice's Adventures in Wonderland for locations, characters and interactive objects suitable for a smaller game. (v0.1-v0.2)
- Defined the goal as recovering four lost treasures, returning them to a safe room and entering the tiny door. (v0.3)
- Reduced the adaptation to a connected set of scenes that could be completely documented and tested. (v0.3-v0.4)

1.4.2 World map and external data

- Designed eleven connected locations with the Rabbit Hole as a one-way transition into Wonderland. (v0.4)
- Made the Hall of Doors a safe room with five exits and a stash for treasure storage. (v0.4-v0.5)
- Added long first-visit descriptions and short repeat-visit descriptions to locations.txt. (v0.5)
- Added sixteen items to items.txt, including four treasures and several usable food, size and protection items. (v0.5-v0.6)
- Added eight characters with descriptions, dialogue, friendly states and random encounter text to characters.txt. (v0.6)
- Added canonical actions, command aliases and help descriptions to actions.txt. (v0.6)
- Added startup validation for missing files, malformed records and invalid location references. (v1.1-v1.2)

1.4.3 Object-oriented class design

- Created Game as the highest class and owner of the complete game state and execution loop. (v0.7)
- Created Location and Locations classes to store, load and search the room map. (v0.7)
- Created Item and Items classes to maintain one location state for every object. (v0.7-v0.8)
- Created Character and Characters classes for fixed characters and random encounters. (v0.8)
- Created Action and Actions classes to load valid commands and aliases. (v0.8)
- Created Inventory with a six-item capacity so the safe room has a practical purpose. (v0.8-v0.9)
- Created Control to read, normalize and parse player input before Game executes an action. (v0.9)

1.4.4 Command parser and game logic

- Added lowercase conversion, punctuation removal and whitespace normalization for all player commands. (v0.9)
- Added support for go north, north and n as equivalent movement commands. (v0.9-v1.0)
- Used longest-alias matching so multiword actions such as pick up are recognized correctly. (v1.0)
- Added object and target parsing for commands such as throw cup at hatter and use fan on queen. (v1.0)
- Added movement, look, take, drop, stash, use, talk, examine, throw, score, help and quit actions. (v1.0-v1.1)
- Added first-visit scoring, inventory capacity, energy, courage and safe-room treasure tracking. (v1.1)
- Added random character encounters and increased the chance of meeting the Queen while carrying the tart. (v1.1)
- Added a fixed random seed and no-random mode for repeatable scripted testing. (v1.1)

1.4.5 Testing and corrections

- Compiled all source files with C++17, Wall, Wextra and pedantic warning flags. (v1.2)
- Created a deterministic command transcript that collects and stashes all four treasures and reaches the winning room. (v1.2)
- Found that command normalization changed underscore-based internal IDs into spaces and bypassed final-door state checks. (v1.2)
- Separated internal identifier handling from player-command normalization so IDs retain their underscores. (v1.2-v1.3)
- Corrected target extraction for use object with target because with and on use different separator lengths. (v1.3)
- Verified the final door remains locked without four stashed treasures and rejects Alice when she is not small enough. (v1.3)
- Verified the complete winning route ends in the Quiet Garden and prints the final score and turn count. (v1.3)
- Added parser, missing-data, inventory-capacity, hostile-character and cross-reference validation cases to the test plan. (v1.3-v1.4)

1.5 Sources and references

1. cppreference.com. (n.d.). C++ reference. Retrieved Jan 2026 - Aug 2026, from <https://cppreference.com/cpp>
 2. Braunschweig, D., & Busbee, K. L. (2018). Programming fundamentals: A modular structured approach (2nd ed.). Rebus Community. <https://press.rebus.community/programmingfundamentals/>
 3. Hansen, J. A. (2013). The Rook's guide to C++. Rook's Guide Press. <https://rooksguide.org/>
 4. Downey, A. B. (1999). Think C++ (Version 1.1.0). Green Tea Press. <https://www.greenteapress.com/thinkcpp/>
 5. leetcode.com. (n.d.). *LeetCode*. Retrieved Jan 2025 - Aug 2026, from <https://leetcode.com/>
 6. Project Gutenberg, *Alice's Adventures in Wonderland*:
<https://www.gutenberg.org/ebooks/11>
1. Ubuntu Manpages, advent and the Open Adventure parser description:
<https://manpages.ubuntu.com/manpages/bionic/man6/advent.6.html>
 1. Eric S. Raymond, *A Brief History of Colossal Cave Adventure*:
<https://www.catb.org/~esr/open-adventure/history.html>

1. Wikipedia, *Colossal Cave Adventure*, used for a broad gameplay and history (the assignment overview before confirming parser details with the manual page): https://en.wikipedia.org/wiki/Colossal_Cave_Adventure